

Sentimentrix – A Sentimental Analysis of Reddit Top 200 Subreddits

Maharshii Patel
251353283
mpate588@uwo.ca

Jiwon Lee
251342617
jlee3859@uwo.ca

Eric Jean
251372435
ejean2@uwo.ca

Jacob Lee
251362845
jlee3887@uwo.ca

Abstract—Online discourse, automated bots and AI-generated content can change the perceived consensus, making it more difficult to accurately display the public mood online. To solve this issue, this research uses data from Reddit, a platform where community voting and moderation help reveal real human interaction, to build a solid sentiment analysis framework.

From the top 200 subreddits (Reddit Top 200), we created a dataset that includes about 40,000 posts, one million comments, and the photos corresponding to them. Our approach includes a detailed pretreatment pipeline that cleans text and incorporates visual content by using the BLIP model to translate photos into text descriptions. As a result, a single multimodal input is produced. Next, we will train a model for sentiment categorization using feature engineering techniques, such as TF-IDF and Reddit-specific information.

Our results demonstrate that this method is capable of reliably identifying the emotions present in various online groups. This provides a scalable method to filter out the noise and obtain a clearer picture of true public sentiment for anyone attempting to understand how people truly feel, including researchers, marketers, and even politicians.

I. INTRODUCTION

It is often difficult to understand the general, unbiased sentiment of the public in situations such as online forums, social media and reviews where opinions can be highly polarized by extremist voices. This often results in some significantly biased feedback. A major problem with online sentiment is that it's very easy to automate user engagement through bots and GenAI. This makes it difficult to trust the overall sentiment expressed across platforms.

In this paper, we decided to visit 200 of Reddit' subreddits, as they have a great moderation system that keeps spam and junk to a minimum and they have an upvote/downvote system which keeps the community engagement rich with user interactions. We will walk through our process of collecting a massive dataset from these subreddits, cleaning them up and applying Natural Language Processing (NLP) techniques to gauge sentiment. By processing a huge number of posts and comments within the subreddit, our model aims to classify emotions more accurately and generally than manual analysis ever could.

Our goal was to see if we could use a machine learning model to accurately “read the room” in these communities. Can a computer program, trained to understand language, correctly identify the general emotional pulse of a discussion thread? To address this question, we structured our paper into 3 sections: Introduction (present problem statement and briefly summarize our contribution), Related Work (compare existing approaches and show our limitations in our implementation) and Methodology (showcase our steps).

II. RELATED WORK (OR LITERATURE REVIEW)

A. Existing Approaches

Sentimental analysis has evolved significantly over the years, with various techniques trying to classify emotions, three prominent groups can be broadly categorized. These include rule-based, machine learning and hybrid approaches [1]. Each of these methods has its strengths, challenges and application domains, making them suitable for different tasks and problems to solve [2], [3].

- 1) **Rule-Based Approaches**
- 2) **Machine Learning Approaches**
- 3) **Hybrid Approaches**

B. Rule-Based Approach

Rule-based methods often rely on predefined linguistic rules and/or lexicons to make decisions on the sentimental outcome by assigning sentiment scores to words or detecting specific linguistic patterns. Their very easy to implement and don't need large datasets for training. However they struggle with ambiguous language (sarcasm/irony).

C. Machine Learning Approach

Both machine learning and deep learning methods are especially good at sentimental analysis. Traditional machine learning techniques like Naive Bayes (NB) and Support Vector Machines (SVM) are very good at classification tasks, especially when the data is well labeled but it's very time consuming to label them properly and they often struggle with evolving words/slangs. Deep learning, on the other hand, excel

at capturing context and long term dependencies within text. Techniques like Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs) and Transformers are highly effective at handling complex sentiments however they require a large amounts of data and is very resource intensive to train.

D. Hybrid Approach

The hybrid method combines both rule-based and machine learning techniques to create a powerful model. These usually includes Ensemble Models which combines multiple machine learning classifiers to improve performance through majority voting or other strategies. Usually this achieves a better performance, when it comes to edge cases, but it can be hard to implement and can be difficult to scale up.

III. METHODOLOGY / PROPOSED METHOD

A. Data Collection & Data Construction

1) *Web Scraping Architecture*: We constructed our dataset using Python with PRAW (Python Reddit API Wrapper). Reddit’s official API wrapper that handles authentication and rate limiting. OAuth credentials were stored securely using environment variables (.env file). PRAW provides a rate limit of 1,000 requests per 600 seconds, which we utilized to maximize data collection efficiency.

2) *Data Collection Strategy*: We targeted the top 200 most popular subreddits to ensure diverse, unbiased community representation. For each subreddit, we collected the top M "hot" posts. This takes advantage of the fact that Reddit’s algorithm prioritizes recent posts with high engagement velocity. For each post, we extracted the top X comments and Y replies to each comment using depth-first search (DFS) traversal of the comment tree structure.

We also identified and cataloged image URLs from multiple sources: direct post URLs, Reddit galleries, and embedded links within post and comment text. Image detection used regex pattern matching for common image hosting domains (i.redd.it, imgur.com, preview.redd.it) and standard image extensions.

3) *Dataset Output Format*: The scraper generates three CSV files per subreddit with the following columns/features:

- Posts: post ID, title, content, score, upvote ratio, author, timestamp, comment count, and image presence flags
- Comments: comment ID, parent ID, text, score, author, timestamp, and embedded image URLs
- Images: image URL, source (post/gallery/comment), image type, and parent linkage

The relational structure via ID fields enables reconstruction of complete discussion threads and post-comment-image relationships.

4) *Dataset Coverage*: The final dataset comprises $\sim 40,000$ posts, $\sim 1,000,000$ comments and $\sim 21,000$ image URLs across the top 200 diverse subreddits, ensuring the model generalizes across domain-specific language patterns and community norms.

B. Data Processing

Our preprocessing pipeline transforms raw data into numerical representations suitable for sentiment analysis models. This process will consist of three main stages: text cleaning, multimodal integration and feature extraction.

1) *Text Cleaning & Normalization*: Raw text will undergo cleaning to remove noise and ensure consistency:

- Reddit-specific formatting removal: Strip markdown syntax, subreddit links (r/subreddit), user mentions (u/username), and Reddit’s custom formatting characters
- URL and special character handling: Remove hyperlinks and special characters while preserving emoticons and punctuation that carry sentiment (e.g., "!", "?")
- Tokenization: Split text into word-level tokens using NLTK or spaCy tokenizers
- Lowercasing: Convert all text to lowercase for uniformity
- Stop word removal: Remove common English stop words (e.g., "the", "and", "is") that don’t contribute to sentiment while preserving negation words (e.g., "not", "never") which are critical for sentiment reversal
- Lemmatization: Reduce words to their base form using WordNetLemmatizer [4] from NLTK to consolidate semantically similar terms (e.g., convert "running" to "run")

2) *Multimodal Integration: Image-to-Text Conversion*: To convert our image content into text content, we will employ vision-language models (VLMs) to convert images into text. We will utilize BLIP (Bootstrapping Language-Image Pre-training) [5]. It’s an image captioning model that generates synthetic captions from images. The reason why BLIP was chosen for its:

- Dual capability: Can handle both conditional and unconditional image captioning, allowing flexible context generation
- Noise filtering: Uses a captioner-filter architecture to generate high-quality synthetic captions while removing noisy descriptions
- Great Documentation: Github repository includes a demo which can be directly taken and used with our images

For each image cataloged in our dataset, we pass it through BLIP’s image encoder to generate a caption. Then, we append the generated caption to the original post or comment text with a delimiter (e.g., "[IMAGE_CONTEXT: caption]")

3) *Feature Extraction & Vectorization*: The cleaned text is converted into numerical features for model input using the following:

- Tokenization: Use BERT tokenizer [6] or similar tokenization to handle uncommon words and domain-specific Reddit slang
- Embeddings: Convert tokens into vector representations using pre-trained embeddings (Word2Vec, GloVe) or contextual embeddings
- Sentiment lexicons: Extract polarity scores from sentiment lexicons, such as VADER [7], as an additional feature
- Metadata: Include metadata features such as upvote ratios, comment depth in reply chains, and timestamp information

C. Feature Engineering & Selection

1) *Text-Based Feature Extraction* : We will employ TF-IDF (Term Frequency-Inverse Document Frequency) as our primary feature extraction technique. TF-IDF captures the importance of terms relative to the collection of documents while downweighting common words across documents.

For contextual understanding, we generate n-gram features to capture sentiment patterns in phrases such as negative contexts ("not good", "don't like"). We will also extract sentiment lexicon scores using VADER (Valence Aware Dictionary and sEntiment Reasoner), which is effective for social media text.

2) *Reddit-Specific Metadata Features*: To leverage Reddit's unique engagement signals, we extract metadata features that correlate with sentiment intensity:

- Post-level features: Upvote ratio, total score (ups), number of comments, awards received, and content type (text vs. image)
- Comment-level features: Comment score, comment depth in reply chain, reply count, and parent-child relationships
- Temporal features: Timestamp information to capture trends over time
- Community features: Subreddit flair text and community-specific context indicators

These metadata features complement textual features by providing implicit sentiment signals through user engagement patterns.

3) *Feature Selection Techniques*: To reduce dimensionality and improve model performance, we will apply feature selection methods:

- Chi-square test: Select features with the highest statistical dependence on sentiment labels
- L1 regularization: Use Lasso regression to automatically eliminate irrelevant features during training

We will retain the top K features (determined via cross-validation) that maximize classification accuracy while minimizing overfitting.

4) *Final Feature Vector* : The final feature representation concatenates TF-IDF vectors, n-gram features, sentiment lexicon scores, and normalized metadata features into a unified feature vector.

D. Model Selection & Architecture

We implement a multi-model approach with traditional machine learning baselines and a transformer-based model to compare performance across different complexities.

1) Baseline Models:

a) *Naive Bayes*: Serves as our simplest baseline using probabilistic classification based on Bayes' theorem. Fast to train and interpretable, it provides a lower-bound performance benchmark. *Support Vector Machine (SVM)*: Our primary baseline ML model. SVM finds the best boundary that separates sentiment classes with the largest possible gap in a space. We will use hyperparameter tuning by doing Grid search with 10-fold cross-validation to optimize C (regularization parameter) and γ (kernel coefficient).:

b) *SVM is well-suited for our task because it handles high-dimensional TF-IDF vectors well and is robust to outliers via soft-margin classification, and achieves 85-90% accuracy on sentiment tasks.*:

2) *Primary Model: Fine-Tuned BERT*: BERT (Bidirectional Encoder Representations from Transformers): Our primary model leverages contextual embeddings to capture nuanced sentiment patterns that traditional models miss. We will use *BertForSequenceClassification* from Hugging Face Transformers and finetune it.

BERT achieves state-of-the-art performance (F1 $\approx$ 0.94 on benchmark datasets) by capturing bidirectional context, handling negation effectively, and understanding social media slang better than traditional models.

E. Training Strategy

1) *Data Splitting*: We split our dataset using stratified sampling to maintain class distribution across train, validation, and test sets. The stratification ensures balanced representation of sentiment classes in each split, preventing bias in model evaluation. Our split ratio is 70% training, 15% validation, 15% test, providing sufficient training data while reserving adequate validation and test data for hyperparameter tuning and unbiased performance evaluation.

2) *SVM Training*: For SVM, we use grid search with 5-fold cross-validation to optimize hyperparameters: regularization parameter C and kernel coefficient γ for RBF kernel. Training uses TF-IDF feature vectors as input, with linear kernel as the primary choice for computational efficiency and Bag-of-Words interpretability. The best cross-validation parameters are stored and applied to train the final SVM model on the full training set.

3) *BERT Fine-Tuning*: For BERT, we use AdamW optimizer with hyperparameters obtained from the following method: Fine-tuning strategy:

- Model initialization: Load pre-trained BERT weights trained on large text corpora
- Classification head: Add a linear layer on top of BERT's token output for binary/multi-class sentiment classification
- Layer freezing: Initially freeze lower BERT layers to prevent catastrophic forgetting, then progressively unfreeze layers during training
- Training configuration: Use AdamW optimizer with learning rate $2e-5$, batch size 16, and 3-5 epochs to prevent overfitting

Validation loss is monitored after each epoch; training stops early if validation loss doesn't improve for 2 consecutive epochs to prevent overfitting.

4) *Label Generation*: Since data is unlabeled, we use VADER sentiment lexicon to automatically generate initial labels for training. VADER scores are normalized to create binary (positive/negative) or multi-class (positive/neutral/negative) labels, which serve as pseudo-labels for model training.

F. Evaluation Metrics & Validation

1) *Primary Metrics*: We use accuracy, precision, recall, and F1-score to evaluate model performance. Accuracy measures overall correctness but can be misleading on imbalanced datasets. Precision quantifies correct positive predictions, minimizing false positives. Recall measures the model's ability to identify all positive cases, minimizing false negatives. F1-score, the harmonic mean of precision and recall, provides balanced performance evaluation particularly suited for imbalanced sentiment classes common in Reddit data.

2) *Confusion Matrix Analysis*: We construct confusion matrices for both SVM and BERT models to analyze error patterns. The confusion matrix breaks down predictions into true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), enabling detailed error analysis to understand which sentiment classes are frequently misclassified.

3) *Cross-Validation Strategy*: We employ 5-fold stratified cross-validation to ensure each fold maintains the same sentiment class distribution as the original dataset. Stratified cross-validation prevents information leakage between folds and provides robust performance estimates, particularly important for imbalanced datasets.

4) *Baseline Comparison*: We compare SVM and BERT performance against the Naive Bayes baseline to quantify improvements from more sophisticated approaches. We also report performance differences between models trained on VADER-labeled data versus manually-annotated validation

subsets to assess label quality impact.

REFERENCES

- [1] Y. Mao, Q. Liu, and Y. Zhang, "Sentiment analysis methods, applications, and challenges: A systematic literature review," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 36, no. 4, p. 102048, 2024, doi: 10.1016/j.jksuci.2024.102048.
- [2] IBM, "What is sentiment analysis?" *IBM*. [Online]. Available: <https://www.ibm.com/think/topics/sentiment-analysis>
- [3] Wikipedia, "Sentiment analysis," *Wikipedia*, [Online]. Available: https://en.wikipedia.org/wiki/Sentiment_analysis.
- [4] NLTK, "WordNetLemmatizer," *NLTK*. [Online]. Available: <https://www.nltk.org/api/nltk.stem.WordNetLemmatizer.html>.
- [5] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," *arXiv*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.12086>.
- [6] P. Kashyap, "Guide to tokenization and padding with BERT: Transforming text into machine-readable data," *Medium*, [Online]. Available: <https://medium.com/@piyushkashyap045/guide-to-tokenization-and-padding-with-bert-transforming-text-into-machine-readable>
- [7] C. Hutto, "vaderSentiment," *GitHub*, [Online]. Available: <https://github.com/cjhutto/vaderSentiment>.